B: Advertising and interactions

Problem statement and worked resolution

Companion to `B_advertising.ipynb`

How to use this explanation. This explanation follows the comparison of four prediction rules, the interpretation of their coefficients, and the independent evaluation. Section numbers follow the notebook's numbered sections where present. Numerical values are rounded and refer to the recorded run, not universal performance claims.

Problem and definitions

The 200-market Advertising data contain TV, radio, and newspaper budgets, each in thousands of dollars, and sales in thousands of units. Predict sales from budgets. The substantive question is whether allowing TV and radio to interact improves prediction among the proposed candidates. Let T, R, N be the three budgets, y observed sales, and $\hat{y}$ predicted sales. An intercept b and slopes w are learned coefficients. Cross-validation (CV) compares prediction rules using repeated held-out subsets of the training rows. Root mean squared error (RMSE) measures prediction error in the same units as sales.

1 Load data and reserve the splits

The notebook reads the CSV from the official textbook site, using a 20-second network timeout. The URL is `https://www.statlearning.com/s/Advertising.csv`. The first CSV column is an exported row index; `index_col=0` prevents treating it as an explanatory variable.

Main Python calls

`urlopen` retrieves the file; `pd.read_csv` builds a data table; `train_test_split` creates reproducible disjoint subsets. Shape, column, missing-value, and index checks verify the expected input. With split seed 23, use 120 training, 40 validation, and 40 test markets. Training selects and fits models; validation audits the choice; test assesses the frozen model. No upload is needed, but the dataset-loading cell requires internet access.

Create the product feature row by row:

```
data["TV_radio"] = data["TV"] * data["radio"]
```

For $T = 100$ and $R = 20$, the product is 2,000. This deterministic feature construction learns no sample parameters and is safe before splitting. All coefficient estimation happens after splitting. Training scatterplots are exploratory views, not test evaluations.

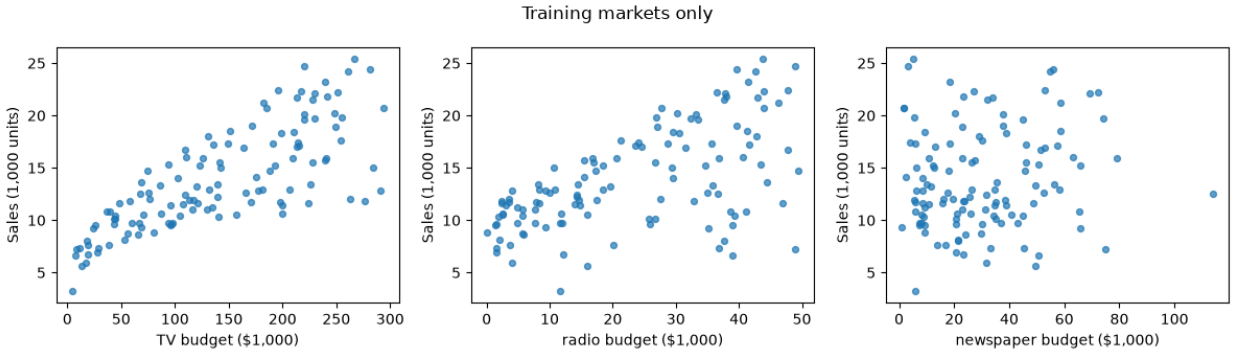


Figure 1: Training sales plotted against each budget. A one-feature view does not hold the other budgets fixed.

2 Compare the four prespecified candidates

Candidate	Prediction rule
Mean baseline	$\hat{y} = \bar{y}_{\text{fit}}$
TV only	$\hat{y} = b + w_T T$
Additive	$\hat{y} = b + w_T T + w_R R + w_N N$
TV–radio interaction	$\hat{y} = b + w_T T + w_R R + w_{TR} TR$

The baseline ignores features. Additive slopes do not depend on other budgets. The interaction model lets the TV slope depend on radio spending, while retaining both main effects. It is still linear regression because it is linear in the coefficients, although a column is nonlinear in the original inputs. The interaction candidate omits newspaper, unlike the additive candidate. Thus the comparison changes the feature set; it does not isolate adding one term to otherwise identical models.

Main Python calls

`LinearRegression` minimizes squared residuals; `KFold` supplies row positions, not fitted models; `.iloc` selects those positions; `mean_squared_error` and `np.sqrt` compute RMSE. The dictionary `candidates` maps names to feature columns. The call `KFold(n_splits=5, shuffle=True, random_state=23)` divides 120 rows into five groups of 24. For each candidate:

1. Fit a new model on four folds, 96 rows.
2. Predict the remaining 24 rows and calculate $\sqrt{\sum_i (y_i - \hat{y}_i)^2 / 24}$.
3. Repeat for all five held-out folds and average their RMSE values.

For the constant baseline, each fold uses only its 96 fitting targets to calculate the mean. All candidates use the same folds, so the comparison is paired by held-out markets. The validation and test sets remain unused.

Candidate	Mean CV RMSE	Fold standard deviation
Mean baseline	4.8636	0.5601
TV only	3.1438	0.2716
Additive	1.5533	0.3446
TV–radio interaction	0.8107	0.2016

All errors are in thousands of sales units. The selected model is the interaction model. Fold standard deviation describes variation, not a confidence interval, and the winning CV score has participated in selection. Finally, refit each regression candidate on all 120 training rows for interpretation and comparison. The chosen full-training fit supplies validation and test predictions; the fold coefficients are not averaged. No scaling is necessary for this ordinary least-squares comparison, and slopes remain in original units.

3 Interpret the interaction with the correct units

The fitted interaction model is approximately

$$\hat{y} = 6.900485 + 0.018634T + 0.032393R + 0.001075TR.$$

`intercept_` contains the constant and `coef_` the slopes in column order. At fixed radio budget $R = r$,

$$\hat{y}(T, r) = (b + w_R r) + (w_T + w_{TR} r)T.$$

Therefore an extra unit of T, meaning an extra \$1,000 of TV budget, changes predicted sales by

$$\hat{y}(T + 1, r) - \hat{y}(T, r) = w_T + w_{TR} r.$$

This quantity is in thousands of sales units; multiply by 1,000 to report units sold.

Radio budget	Change in thousands of units	Change in units
\$0	0.018634	18.6
\$30,000	0.050899	50.9

The TV coefficient alone is the slope at zero radio spending. Saying that every \$1,000 of TV always adds 18.6 predicted units ignores the interaction. Symmetrically, the radio slope is $w_R + w_{TR}T$.

A positive product coefficient means the two budgets reinforce each other in the fitted prediction. It does not establish a causal return on spending: markets were not randomized to budgets. The intercept predicts sales with zero TV and radio spending; that combination may extrapolate outside well-supported observations.

4 Inspect residuals and audit validation

A residual is $e_i = y_i - \hat{y}_i$. A positive residual means underprediction; for example $e_i = 2$ means underpredicting by 2,000 units. The plots compare residuals against fitted sales on training markets. A curved pattern suggests a missing relationship; changing spread suggests errors whose scale depends on the fitted level. Training residuals averaging zero is expected from least squares with an intercept and does not, by itself, demonstrate good prediction.

Main Python calls

`candidate_prediction(name, frame)` applies the already fitted model or repeats the training mean. It does not call `fit`. The validation calculation uses it on the 40 reserved markets. The selected interaction model has validation RMSE about 1.3836 thousand units. This differs from its mean CV score because the scoring rows differ and the full-training model used 120 fitting rows instead of 96. A small validation set can give a noticeably different error. Under this protocol the audit does not reopen candidate selection.

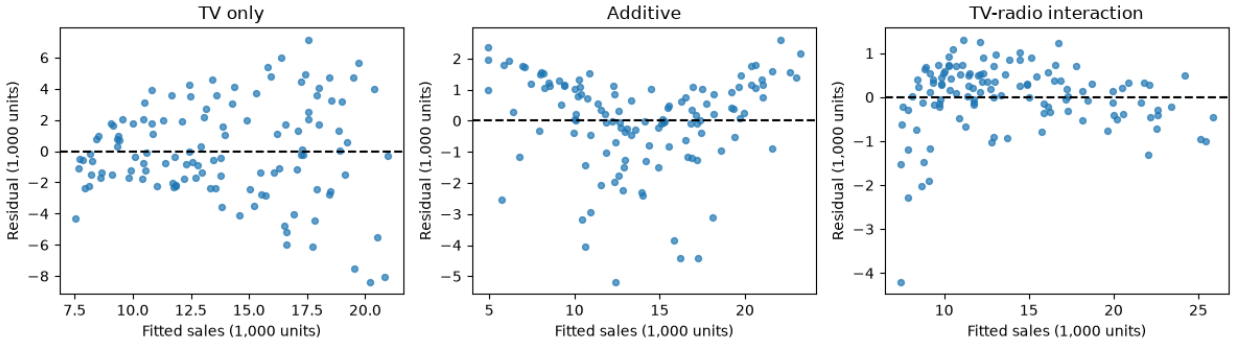


Figure 2: Training residuals for the three regression models. These plots diagnose fitted relationships; they are not estimates of error on unseen markets.

5 Evaluate once on test data

For m test markets define

$$\text{MAE} = \frac{1}{m}\sum_i |y_i - \hat{y}_i|, \quad \text{RMSE} = \sqrt{\frac{1}{m}\sum_i (y_i - \hat{y}_i)^2}, \quad R^2 = 1 - \frac{\sum_i (y_i - \hat{y}_i)^2}{\sum_i (y_i - \bar{y}_{\text{test}})^2}.$$

MAE averages absolute errors; RMSE penalizes large errors more strongly. Both are in thousands of units. R^2 is dimensionless and can be negative. It is not a percentage of individually correct predictions.

Model	MAE	RMSE	R^2
Training-mean baseline	4.6325	5.6526	−0.0176
Selected interaction	0.6501	0.8531	0.9768

The interaction predictions are off by about 650 units on average in absolute value. Their squared error is about 2.3% of the test-mean reference error. The baseline predicts the *training* mean, whereas the R^2 denominator uses the *test* mean only to score predictions. This explains the baseline's slightly negative R^2 without implying any test-target leakage.

Resolution of the interpretation questions

The interaction wins the specified CV comparison and improves greatly over the constant baseline on test data. The TV slope increases with radio budget. Residual patterns can motivate future models, but none should be introduced after examining this final test and then evaluated as if it were untouched. Estimated coefficients need not match the textbook's full-data fit because only 120 rows were fitted here. Finally, a random market split addresses prediction for similarly sampled markets; it establishes neither future-period performance nor causal advertising effects.

Further explanation: interactions and honest comparison

What a product feature adds to the model

For an additive TV/radio model, the predicted benefit of a TV increase is the same at every radio budget. An interaction adds one specific departure from that restriction, not arbitrary nonlinearity. For TV change ΔT and radio change ΔR, consider the difference between two TV increments:

$$[\hat{y}(T + \Delta T, R + \Delta R) - \hat{y}(T, R + \Delta R)] - [\hat{y}(T + \Delta T, R) - \hat{y}(T, R)] = w_{TR}\Delta T \Delta R.$$

This double difference is zero in an additive model. Here $\Delta T = 10$ and $\Delta R = 5$ mean TV spending increases by \$10,000 and radio spending by \$5,000; the extra interaction contribution is approximately $0.001075 \times 10 \times 5 = 0.05375$ thousand units, or 53.75 units. This is a transparent way to interpret the coefficient without confusing its units with a main-effect slope.

Why centering changes coefficients but not fitted predictions

Suppose we replace T, R by $T' = T - T_0$ and $R' = R - R_0$, retaining both main effects and their product. Expanding the original equation gives

$$\hat{y} = b' + (w_T + w_{TR}R_0)T' + (w_R + w_{TR}T_0)R' + w_{TR}T'R'.$$

The new intercept b' is the old prediction at (T_0, R_0). The new TV slope is the slope at radio budget R_0, which may be more meaningful than the slope at zero. The set of prediction functions is unchanged for unpenalized fitting, yet the coefficient table changes. This is why interpreting a coefficient requires knowing both the model terms and the coordinate system. Dropping the main effects would break this simple invariance.

Two reasonable error averages are not the same statistic

The notebook averages five fold RMSE values. Another possible report would combine all held-out squared errors, then take one square root. Even with equal fold sizes these differ:

$$\frac{1}{5}\sum_{k=1}^{5}\sqrt{\mathrm{MSE}_k} \quad \text{versus} \quad \sqrt{\frac{1}{5}\sum_{k=1}^{5}\mathrm{MSE}_k}.$$

For two illustrative fold MSE values 1 and 9, the first convention gives 2 and the second $\sqrt{5} \approx 2.236$. Neither is a coding error when stated in advance. Do not expect a metric recomputed after pooling predictions to reproduce a mean-of-folds table exactly. Also, the five fits share many training rows: their scores are not five independent repetitions from which an ordinary confidence interval automatically follows.

What the comparison does and does not isolate

The additive candidate includes newspaper, while the interaction candidate drops it and adds TV times radio. A lower error therefore supports the latter *prediction rule*, but cannot attribute the whole improvement to the interaction alone. A controlled feature comparison would evaluate TV plus radio against TV plus radio plus their product, using a prespecified new experiment. Comparing features repeatedly until the held-out score improves introduces selection optimism.

Using documentation and avoiding silent mistakes

The LinearRegression documentation explains the input shapes and learned coefficients; `coef_` follows the supplied feature-column order. The common-pitfalls guide explains why evaluation rows must receive the same preprocessing as fitting rows. Here this includes recomputing the product in the same budget units: dollars instead of thousands of dollars would change its numerical scale by a factor of a million.

Practical tips. Print input columns beside coefficients, check the units of every feature interaction, and keep the exported CSV index out of the feature list. When RMSE looks unexpectedly large, first check whether sales are in units or thousands of units. Model interpretation at extreme budgets requires data support; an algebraically valid prediction may still be poor extrapolation. A high R^2 does not settle these questions.